

Chapters 9 and 10: Hierarchical models and custom distributions

Shravan Vasishth

2026-03-26

Contents

1	<i>Textbook</i>	2
2	<i>Varying intercept model with Stan</i>	2
3	<i>Uncorrelated varying intercept and slopes model with Stan</i>	7
4	<i>Aside: A simple non-centered parameterization</i>	11
5	<i>Back to the hierarchical model with uncorrelated random effects</i>	12
6	<i>Correlated varying intercept varying slopes model</i>	16
7	<i>Aside: Cholesky factorization</i>	19
7.1	<i>Cholesky factorization in R</i>	21
7.2	<i>Application of the Cholesky factorization in Stan</i>	23
8	<i>By-subject and by-items correlated varying intercept varying slopes model</i>	27
9	<i>Custom distributions in Stan</i>	29
9.1	<i>A change of variables with the reciprocal normal distribution</i>	30
9.2	<i>Scaling a probability density with the Jacobian adjustment</i>	33
9.3	<i>Validation of a computed posterior distribution</i>	39

1 Textbook

Introduction to Bayesian Data Analysis for Cognitive Science

Nicenboim, Schad, Vasishth

- Online version: <https://bruno.nicenboim.me/bayescogsci/>
- Source code: <https://github.com/bnicenboim/bayescogsci>
- Physical book: [here](#)

Be sure to read the textbook's chapter 8 in addition to watching this lecture.

2 Varying intercept model with Stan

Recall that earlier we fit models to investigate the effect of cloze probability on EEG averages in the N400 spatiotemporal time window. For our first model, we'll make the (implausible) assumption that only the average signal varies across subjects, but all subjects share the same effect of cloze probability. This means that the likelihood incorporates the assumption that the intercept, α , is adjusted with the term u_i for each subject.

$$signal_n \sim Normal(\alpha + u_{subj[n]} + c_cloze_n \cdot \beta, \sigma) \quad (1)$$

$$\alpha \sim Normal(0, 10)$$

$$\beta \sim Normal(0, 10)$$

$$u \sim Normal(0, \tau_u) \quad (2)$$

$$\tau_u \sim Normal_+(0, 20)$$

$$\sigma \sim Normal_+(0, 50)$$

Here n represents each observation, the n th row in the data frame and $subj[n]$ is the subject that corresponds to observation n . We present the mathematical notation of the likelihood with “multiple indexing” [see chapter 19 of the User's guide]: the index of u is provided by the vector $subj$.

Before we discuss the Stan implementation, let's see what the vector μ , the location of the normal likelihood, looks like. There are 2863 observations; that means that $\mu = \{\mu_1, \mu_2, \dots, \mu_{2863}\}$. We have 37 subjects which means that $\mathbf{u} = \{u_1, u_2, \dots, u_{37}\}$. The following equation shows that the use of multiple indexing allows us to have a vector of adjustments with only 37 different elements, with a total length of 2863. In the equation below, the multiplication operator $\circ$ is the Hadamard or elementwise product: when we write $X \circ B$, both X and B have the same dimensions $m \times n$, and each cell in location $[i, j]$ (where $i = 1, \dots, m$, and $j = 1, \dots, n$) in X and B are multiplied to give a matrix that also has dimensions $m \times n$.¹

¹ Whereas in $\mathbf{z}$, α and β are "broadcasted" into a 2863-vector for ease of exposition, Stan uses vector-scalar arithmetic.

$$\begin{aligned}
\boldsymbol{\mu} &= \begin{bmatrix} \mu_1 \\ \mu_2 \\ \dots \\ \mu_{101} \\ \mu_{102} \\ \dots \\ \mu_{215} \\ \mu_{216} \\ \mu_{217} \\ \dots \\ \mu_{1000} \\ \dots \\ \mu_{2863} \end{bmatrix} = \begin{bmatrix} \alpha \\ \alpha \\ \dots \\ \alpha \\ \alpha \\ \dots \\ \alpha \\ \alpha \\ \alpha \\ \dots \\ \alpha \\ \dots \\ \alpha \end{bmatrix} + \begin{bmatrix} u_{subj[1]} \\ u_{subj[2]} \\ \dots \\ u_{subj[101]} \\ u_{subj[102]} \\ \dots \\ u_{subj[215]} \\ u_{subj[216]} \\ u_{subj[217]} \\ \dots \\ u_{subj[1000]} \\ \dots \\ u_{subj[2863]} \end{bmatrix} + \begin{bmatrix} ccloze_1 \\ ccloze_2 \\ \dots \\ ccloze_{101} \\ ccloze_{102} \\ \dots \\ ccloze_{215} \\ ccloze_{216} \\ ccloze_{217} \\ \dots \\ ccloze_{1000} \\ \dots \\ ccloze_{2863} \end{bmatrix} \circ \begin{bmatrix} \beta \\ \beta \\ \dots \\ \beta \\ \beta \\ \dots \\ \beta \\ \beta \\ \beta \\ \dots \\ \beta \\ \dots \\ \beta \end{bmatrix} \\
&= \begin{bmatrix} \alpha \\ \alpha \\ \dots \\ \alpha \\ \alpha \\ \dots \\ \alpha \\ \alpha \\ \alpha \\ \dots \\ \alpha \\ \dots \\ \alpha \end{bmatrix} + \begin{bmatrix} u_1 \\ u_1 \\ \dots \\ u_2 \\ u_2 \\ \dots \\ u_3 \\ u_3 \\ u_3 \\ \dots \\ u_{13} \\ \dots \\ u_{37} \end{bmatrix} + \begin{bmatrix} \\ \\ \dots \\ \\ \\ \dots \\ \\ \\ \dots \\ \\ \dots \\ \\ \end{bmatrix} \circ \begin{bmatrix} \beta \\ \beta \\ \dots \\ \beta \\ \beta \\ \dots \\ \beta \\ \beta \\ \beta \\ \dots \\ \beta \\ \dots \\ \beta \end{bmatrix}
\end{aligned} \tag{3}$$

In this model, each subject has their own intercept adjustment u_i , with i indexing the subjects. If u_i is positive, the subject has a more positive EEG signal than the average over all the subjects; if u_i is negative, then the subject has a more negative EEG signal than the average; and if u_i is 0, then the subject has the same EEG signal as the average. As we discussed earlier, since we are estimating α and u at the same time and we assume that the average of the u 's is 0 (since it is assumed to be normally distributed with a mean of zero and there is an intercept to absorb

any non-zero mean), whatever the subjects have in common “goes” to α , and u only “absorbs” the differences between subjects through the variance component τ_u .

This model is implemented in the file `hierarchical1.stan`, available in the `bcogsci` package:

```
data {
  int<lower=1> N;
  vector[N] signal;
  int<lower = 1> N_subj;
  vector[N] c_cloze;
  // The following line defines an array of integers;
  array[N] int<lower = 1, upper = N_subj> subj;
}
parameters {
  real<lower = 0> sigma;
  real<lower = 0> tau_u;
  real alpha;
  real beta;
  vector[N_subj] u;
}
model {
  target += normal_lpdf(alpha | 0, 10);
  target += normal_lpdf(beta | 0, 10);
  target += normal_lpdf(sigma | 0, 50) -
    normal_lccdf(0 | 0, 50);
  target += normal_lpdf(tau_u | 0, 20) -
    normal_lccdf(0 | 0, 20);
  target += normal_lpdf(u | 0, tau_u);
  target += normal_lpdf(signal | alpha + u[subj] +
    c_cloze * beta, sigma);
}
```

In the Stan code above, we use

`array [N] int<lower = 1, upper = N_subj> subj;`
to define a one-dimensional *array* of N elements that contains integers (bounded between 1 and N_subj). As explained earlier, the difference between vectors and one-dimensional arrays is that vectors can only contain real numbers and can be used with matrix algebra functions,

and arrays can contain any type but can't be used with matrix algebra functions. We use `normal_lpdf` rather than `normal_glm_lpdf` since at the moment there is no efficient likelihood implementation of hierarchical generalized linear models.

The following code centers the predictor `cloze` and stores the data required by the Stan model in a list. Because we are using `subj` as a vector of indices, we need to be careful to have integers starting from 1 and ending in `N_subj` without skipping any number (but the order of the subject ids won't matter).²

```
data("df_eeg")
df_eeg <- df_eeg %>%
  mutate(c_cloze = cloze - mean(cloze))
ls_eeg <- list(
  N = nrow(df_eeg),
  signal = df_eeg$n400,
  c_cloze = df_eeg$c_cloze,
  subj = df_eeg$subj,
  N_subj = max(df_eeg$subj)
)
```

² It often happens that the subject ids are not in sequence in a given data frame. This can happen when, for example, one has lost some subjects due to incomplete data or for some other reason. In such situations, we can re-number the subjects sequentially as follows: type `as.numeric(as.factor(df_eeg$subj))` to transform a vector where some numbers are skipped into a vector with consecutive numbers. For example, both `as.numeric(as.factor(c(1, 3, 4, 7, 9)))` and `as.numeric(as.factor(paste0("subj", c(1, 3, 4, 7, 9))))` will give as output 1 2 3 4 5.

Fit the model:

```
hierarchical1 <- system.file("stan_models",
                             "hierarchical1.stan",
                             package = "bcogsci")
fit_eeg1 <- stan(hierarchical1, data = ls_eeg)
```

Summary of the model:

```
print(fit_eeg1, pars = c("alpha", "beta", "sigma", "tau_u"))
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##          mean se_mean   sd  2.5%   25%   50%   75%  97.5% n_eff Rhat
## alpha   3.64     0.01 0.42   2.81   3.37   3.64   3.91   4.49  1153    1
## beta    2.31     0.01 0.53   1.27   1.95   2.32   2.67   3.34  5327    1
```

```
## sigma 11.64    0.00 0.15 11.35 11.53 11.63 11.74 11.94 4464    1
## tau_u  2.18    0.01 0.36  1.55  1.93  2.15  2.39  2.96 2244    1
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:38:46 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

3 Uncorrelated varying intercept and slopes model with Stan

In the following model, we relax the strong assumption that every subject will be affected equally by the manipulation. For ease of exposition, we start by assuming that the adjustments for the intercept and slope are not correlated.

$$signal_n \sim Normal(\alpha + u_{subj[n],1} + c_{cloze_n} \cdot (\beta + u_{subj[n],2}), \sigma) \quad (4)$$

$$\begin{aligned} \alpha &\sim Normal(0, 10) \\ \beta &\sim Normal(0, 10) \\ u_1 &\sim Normal(0, \tau_{u_1}) \\ u_2 &\sim Normal(0, \tau_{u_2}) \\ \tau_{u_1} &\sim Normal_+(0, 20) \\ \tau_{u_2} &\sim Normal_+(0, 20) \\ \sigma &\sim Normal_+(0, 50) \end{aligned} \quad (5)$$

We implement this in Stan in `hierarchical2.stan`:

```
data {
  int<lower=1> N;
  vector[N] signal;
  int<lower = 1> N_subj;
  vector[N] c_cloze;
  array[N] int<lower = 1, upper = N_subj> subj;
}
parameters {
  real<lower = 0> sigma;
```

```

vector<lower = 0>[2] tau_u;
real alpha;
real beta;
matrix[N_subj, 2] u;
}
model {
  target += normal_lpdf(alpha | 0, 10);
  target += normal_lpdf(beta | 0, 10);
  target += normal_lpdf(sigma | 0, 50) -
    normal_lccdf(0 | 0, 50);
  target += normal_lpdf(tau_u[1] | 0, 20) -
    normal_lccdf(0 | 0, 20);
  target += normal_lpdf(tau_u[2] | 0, 20) -
    normal_lccdf(0 | 0, 20);
  target += normal_lpdf(u[, 1] | 0, tau_u[1]);
  target += normal_lpdf(u[, 2] | 0, tau_u[2]);
  target += normal_lpdf(signal | alpha + u[subj, 1] +
    c_cloze .* (beta + u[subj, 2]), sigma);
}

```

In the previous model, we assign the same prior distribution to both `tau_u[1]` and `tau_u[2]`, and thus in principle we could have written the two statements in one (we multiply by 2 because there are two PDFs that need to be corrected for the truncation):

```

target += normal_lpdf(tau_u | 0, 20) -
  2 * normal_lccdf(0 | 0, 20);

```

Fit the model as follows:

```

hierarchical2 <- system.file("stan_models",
                             "hierarchical2.stan",
                             package = "bcogsci")
fit_eeg2 <- stan(hierarchical2, data = ls_eeg)

```

```

## Warning: There were 3 chains where the estimated Bayesian Fraction of Missing Information
## https://mc-stan.org/misc/warnings.html#bfmi-low

## Warning: Examine the pairs() plot to diagnose sampling problems

## Warning: The largest R-hat is 1.08, indicating chains have not mixed.

```



```
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#r-hat
```

```
## Warning: Bulk Effective Samples Size (ESS) is too low, indicating posterior means and
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#bulk-ess
```

```
## Warning: Tail Effective Samples Size (ESS) is too low, indicating posterior variances
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#tail-ess
```

We see that there are warnings. As we increase the complexity and the number of parameters, the sampler has a harder time exploring the parameter space.

Show the summary of the model below:

```
print(fit_eeg2, pars = c("alpha", "beta", "tau_u", "sigma"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##               mean se_mean   sd  2.5%  25%   50%   75%  97.5% n_eff Rhat
## alpha         3.61    0.01 0.43  2.73  3.33  3.63  3.89  4.42  1333 1.00
## beta          2.33    0.01 0.61  1.12  1.93  2.32  2.74  3.53  3772 1.00
## tau_u[1]      2.18    0.01 0.36  1.55  1.92  2.14  2.40  2.96  2150 1.00
## tau_u[2]      1.67    0.11 0.91  0.26  0.96  1.67  2.30  3.47    73 1.07
## sigma        11.62    0.00 0.16 11.31 11.51 11.61 11.72 11.93  5074 1.00
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:39:17 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

We see that `tau_u[2]` has a low number of effective samples (`n_eff`).

The traceplots are displayed in Figure 1.

```
traceplot(fit_eeg2, pars = c("alpha", "beta", "tau_u",
```

Figure 1 shows that the chains of the parameter `tau_u[2]` are not mixing properly. This parameter is specially problematic because there are not enough data from each

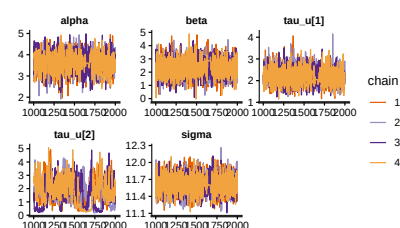


Figure 1: Traceplots of the parameters.

subject to estimate this parameter accurately, its estimated mean is quite small (in comparison with sigma), it's bounded by zero, and there is a dependency between this parameter and $u[, 2]$. This makes the exploration by the sampler quite hard.

Pairs plots can be useful to uncover pathologies in the sampling, since we can visualize dependencies between samples, which are in general problematic.³ The following code creates a pair plot where we see the samples of $\tau_{u[2]}$ against some of the adjustments to the slope u ; see Figure 2.

```
pairs(fit_eeg2, pars = c("tau_u[2]", "u[1,2]", "u[2,2]"
```

Compare with $\tau_{u[1]}$ plotted against the by-subject adjustments to the intercept. In Figure 3, instead of funnels we see blobs, indicating no strong dependency between the parameters.

```
pairs(fit_eeg2, pars = c("tau_u[1]", "u[1,1]", "u[2,1]"
```

In contrast to Figure 3, which shows blob-shaped clouds of samples, Figure 2 shows a relatively strong dependency between the samples of some of the parameters of the model, in particular τ_2 and the samples of $u_{i,2}$. This strong dependency is hindering the exploration of the sampler leading to the warnings we saw in Stan. However, the problem that the sampler faces is, in fact, more serious than what our initial plots show. Stan samples in an *unconstrained* space where all the parameters can range from minus infinity to plus infinity, and then transforms back the parameters to the constrained space that we specified where, for example, a standard deviation parameter is restricted to be positive. This means that Stan is actually sampling from a transformed parameter equivalent to $\log(\tau_{u[2]})$ rather than from $\tau_{u[2]}$. We can use `mcmc_pairs` to see the actual funnel; see Figure 4.

```
mcmc_pairs(as.array(fit_eeg2),
  pars = c("tau_u[2]", "u[1,2]"),
  transform = list(`tau_u[2]` = "log")
)
```

³ See <https://mc-stan.org/misc/warnings.html>.

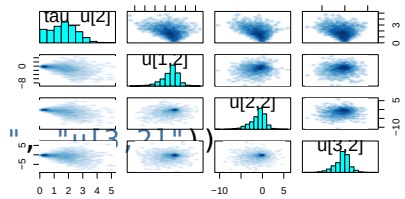


Figure 2: Pair plots showing a relatively strong dependency (funnel-shaped clouds of samples) between the samples of $\tau_{u[2]}$ and some of the by-subject adjustments to the slope.

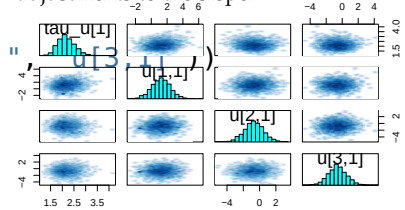


Figure 3: Pair plots showing no strong dependency (blob-shaped clouds of samples) between the samples of $\tau_{u[1]}$ and some of the by-subject adjustments to the intercept.

At the neck of the funnel, $\tau_{u[2]}$ is close to zero (and $\log(\tau_{u[2]})$ is a negative number) and thus the adjustment u is constrained to be near 0. This is a problem because a step size that's optimized to work well in the broad part of the funnel will fail to work appropriately in the neck of the funnel and vice versa; see also Neal's funnel and the optimization chapter of the User's guide (section 25.7 of the User's guide). There are two options: We might just remove the by-subject varying slope since it's not giving us much information anyway, or we can alleviate this problem by reparameterizing the model. In general, this sort of thing is the trickiest and probably most annoying part of modeling. A model can be theoretically and mathematically sound, but still fail to converge. The best advice to solve this type of problem is to start small with simulated data where we know the true values of the parameters, and increase the complexity of the models gradually. Although in this example the problem was clearly in the parameterization of $\tau_{u[2]}$, in many cases the biggest hurdle is to identify where the problem lies. Fortunately, the issue with $\tau_{u[2]}$ is a common problem which is easy to solve by using a non-centered parameterization. The next section explains the specific reparameterization we use for the improved version of our Stan code.

4 *Aside: A simple non-centered parameterization*

The sampler can explore the parameter space more easily if its step size is appropriate for all the parameters. This is achieved when there are no strong dependencies between parameters. We want to assume the following.

$$\mathbf{u}_2 \sim \text{Normal}(0, \tau_{u_2}) \quad (6)$$

where $\mathbf{u}_2$ is the column vector of $u_{i,2}$'s. The index i refers to the subject id.

We can transform a vector v into z-scores as follows.

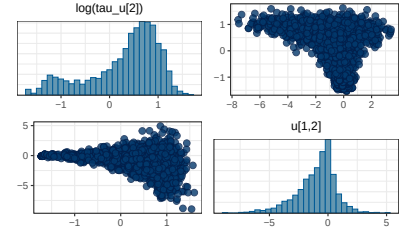


Figure 4: Pair plots showing a strong dependency (funnel-shaped clouds of samples) between the samples of au_2 and one of the by-subject adjustments to the intercept ($u_{1,2}$).

$$\mathbf{z}_v = \frac{\mathbf{v} - \text{mean}(\mathbf{v})}{SD(\mathbf{v})} \quad (7)$$

Transforming the parameter u_2 into z-scores amounts to centering it, so we can call this centered parameterization.

$$\mathbf{z}_{u_2} = \frac{\mathbf{u}_2 - 0}{\tau_{u_2}} \quad (8)$$

where

$$\mathbf{z}_{u_2} \sim \text{Normal}(0, 1) \quad (9)$$

Now $\mathbf{z}_{u_2}$ is easier to sample because it doesn't depend on other parameters (in particular, it is no longer conditional on τ) and its scale is 1. Once we have sampled this centered parameter, we can derive the actual parameter we care about by carrying out the inverse operation, which is called a non-centered parameterization:

$$\mathbf{u}_2 = \mathbf{z}_{u_2} \cdot \tau_{u_2} \quad (10)$$

A question that might be raised here is whether using a non-centered parameterization is always a good idea. Betancourt has pointed out that the extremeness of the dependency depends on the amount of data, and the efficacy of the parameterization depends on the strength of the data (on how informative the data is). When there is enough data, this parameterization is unnecessary and it may be more efficient to use the centered parameterization. However, cases where there is enough data to render this parameterization useless might also be cases where the partial pooling of the hierarchical models may not be needed in the first place. Although data from conventional lab experiments in psychology, psycholinguistics, and related areas seem to benefit from the non-centered parameterization, the jury is still out for larger data sets with thousands of subjects from crowdsourcing websites.

5 Back to the hierarchical model with uncorrelated random effects

From a mathematical point of view, the following model is equivalent to the one described in Equations 4 and 5.

However, as discussed previously, the computational implementation of the “new” model is more efficient. The following model includes the reparameterization of both adjustments u_1 and u_2 , although the reparameterization of u_1 is not strictly necessary (we didn’t see any problems in the trace plots), it won’t hurt either and the Stan code will be simpler.

$$signal_n \sim Normal(\alpha + u_{subj[n],1} + c_{cloze_n} \cdot (\beta + u_{subj[n],2}), \sigma) \quad (11)$$

$$\begin{aligned} \alpha &\sim Normal(0, 10) \\ \beta &\sim Normal(0, 10) \\ z_{u_1} &\sim Normal(0, 1) \\ z_{u_2} &\sim Normal(0, 1) \\ \tau_{u_1} &\sim Normal_+(0, 20) \\ \tau_{u_2} &\sim Normal_+(0, 20) \\ \sigma &\sim Normal_+(0, 50) \\ u_1 &= z_{u_1} \cdot \tau_{u_1} \\ u_2 &= z_{u_2} \cdot \tau_{u_2} \end{aligned} \quad (12)$$

The following Stan code (`hierarchical3.stan`) uses the previous parameterization, and introduces some new Stan functions: `to_vector()` converts a matrix into a long column vector (in column-major order, that is, concatenating the columns from left to right); and `std_normal_lpdf()` implements the log PDF of a standard normal distribution, a normal distribution with location 0 and scale 1. This function is just a more efficient version of `normal_lpdf(... | 0, 1)`. We also introduce a new optional block called `transformed parameters`. With each iteration of the sampler, the values of the parameters (i.e., `alpha`, `beta`, `sigma`, and `z`) are available at the `transformed parameters` block, and we can derive new auxiliary variables based on them. In this case, we use `z_u` and `tau_u` to obtain `u`, that then becomes available in the `model` block.

```
data {
```

```

    int<lower=1> N;
    vector[N] signal;
    int<lower = 1> N_subj;
    vector[N] c_cloze;
    array[N] int<lower = 1, upper = N_subj> subj;
  }
  parameters {
    real<lower = 0> sigma;
    vector<lower = 0>[2] tau_u;
    real alpha;
    real beta;
    matrix[N_subj, 2] z_u;
  }
  transformed parameters {
    matrix[N_subj, 2] u;
    u[, 1] = z_u[, 1] * tau_u[1];
    u[, 2] = z_u[, 2] * tau_u[2];
  }
  model {
    target += normal_lpdf(alpha | 0, 10);
    target += normal_lpdf(beta | 0, 10);
    target += normal_lpdf(sigma | 0, 50) -
      normal_lccdf(0 | 0, 50);
    target += normal_lpdf(tau_u[1] | 0, 20) -
      normal_lccdf(0 | 0, 20);
    target += normal_lpdf(tau_u[2] | 0, 20) -
      normal_lccdf(0 | 0, 20);
    target += std_normal_lpdf(to_vector(z_u));
    target += normal_lpdf(signal | alpha + u[subj, 1] +
      c_cloze .* (beta + u[subj, 2]), sigma);
  }

```

By reparameterizing the model we can also optimize it more, we can convert the matrix `z_u` into a long column vector allowing us to use a single call of `std_normal_lpdf`. Fit the model named `hierarchical3.stan`.

```

hierarchical3 <- system.file("stan_models",
                             "hierarchical3.stan",
                             package = "bcogsci")

```

```
fit_eeg3 <- stan(hierarchical3, data = ls_eeg)
```

Verify that the model worked as expected by printing its summary and traceplots; see Figure 5.

```
print(fit_eeg3, pars = c("alpha", "beta", "tau_u", "sigma"))
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##          mean se_mean   sd  2.5%  25%   50%   75%  97.5% n_eff Rhat
## alpha      3.64    0.01 0.42  2.80  3.37  3.65  3.92  4.46  1743   1
## beta       2.33    0.01 0.62  1.13  1.91  2.34  2.75  3.55  4250   1
## tau_u[1]   2.17    0.01 0.37  1.52  1.91  2.14  2.40  2.97  1392   1
## tau_u[2]   1.73    0.03 0.91  0.10  1.05  1.74  2.40  3.50  1085   1
## sigma     11.61    0.00 0.16 11.31 11.51 11.61 11.72 11.92  6023   1
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:39:50 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
traceplot(fit_eeg3, pars = c("alpha", "beta", "tau_u",
```

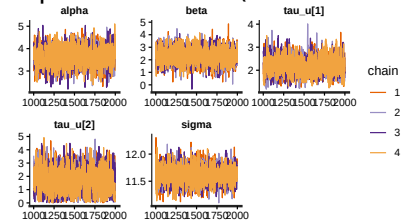


Figure 5: Traceplots of the parameters.

Although the samples of $\tau_u[2]$ are still conditioned by the adjustments for the slope, $u[2]$, these latter parameters are not the ones explored by the model, the auxiliary parameters, z_u , are the relevant ones for the sampler. The plots in Figures 6 and 7 show that although samples from $\log(\tau_u[2])$ and $u[1,2]$ depend on each other, samples from $\log(\tau_u[2])$ and $z_u[1,2]$ do not.

```
mcmc_pairs(as.array(fit_eeg3),
  pars = c("tau_u[2]", "u[1,2]"),
  transform = list(`tau_u[2]` = "log"))
```

```
mcmc_pairs(as.array(fit_eeg3),
  pars = c("tau_u[2]", "z_u[1,2]"),
  transform = list(`tau_u[2]` = "log"))
```

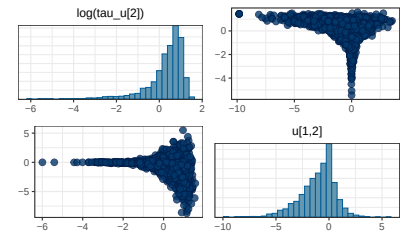


Figure 6: Pair plots showing a clear dependency (funnel-shaped clouds of samples) between the samples of au_2 and some of the by-subject adjustments to the slope.

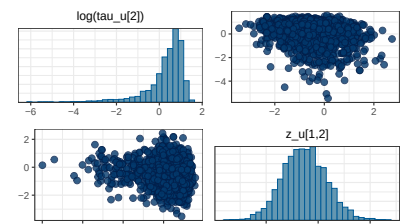


Figure 7: Pair plots showing no clear dependency between the samples of au_2

6 Correlated varying intercept varying slopes model

For the model with correlated varying intercepts and slopes, the likelihood remains identical to the model without a correlation between group-level intercepts and slopes. Priors and hyperpriors change to reflect the potential correlation between by-subject adjustments to intercepts and slopes:

$$signal_n \sim Normal(\alpha + u_{subj[n],1} + c_{cloze_n} \cdot (\beta + u_{subj[n],2}), \sigma) \quad (13)$$

The correlation is indicated in the priors on the adjustments for vector of by-subject intercepts u_1 and the vector of by-subject slopes u_2 .

- Priors:

$$\begin{aligned} \alpha &\sim Normal(0, 10) \\ \beta &\sim Normal(0, 10) \\ \sigma &\sim Normal_+(0, 50) \\ \begin{pmatrix} u_{i,1} \\ u_{i,2} \end{pmatrix} &\sim \mathcal{N} \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Sigma_u \right) \end{aligned} \quad (14)$$

where $i = \{1, \dots, N_{subj}\}$

$$\Sigma_u = \begin{pmatrix} \tau_{u_1}^2 & \rho_u \tau_{u_1} \tau_{u_2} \\ \rho_u \tau_{u_1} \tau_{u_2} & \tau_{u_2}^2 \end{pmatrix} \quad (15)$$

$$\begin{aligned} \tau_{u_1} &\sim Normal_+(0, 20) \\ \tau_{u_2} &\sim Normal_+(0, 20) \\ \rho_u &\sim LKJcorr(2) \end{aligned} \quad (16)$$

As a first attempt, we write this model following the mathematical notation as closely as possible. We'll see that this will be problematic in terms of efficient sampling and convergence. In this Stan model (`hierarchical_corr.stan`), we use some new functions and types:

- `corr_matrix[n] M`; defines a (square) matrix of n rows and n columns called M , symmetrical around a diagonal of ones.


```

    target += normal_lpdf(signal | alpha + u[subj, 1] +
                          c_cloze .* (beta + u[subj, 2]), sigma);
}

```

Problematic aspects of the model before the reparameterization, that is, dependencies between parameters, are also present here. Fit the model as follows:

```

hierarchical_corr <- system.file("stan_models",
                                "hierarchical_corr.stan",
                                package = "bcogsci")
fit_eeg_corr <- stan(hierarchical_corr, data = ls_eeg)

## Warning: There were 1 divergent transitions after warmup. See
## https://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup
## to find out why this is a problem and how to eliminate them.

## Warning: There were 4 chains where the estimated Bayesian Fraction of Missing Information
## https://mc-stan.org/misc/warnings.html#bfmi-low

## Warning: Examine the pairs() plot to diagnose sampling problems

## Warning: The largest R-hat is NA, indicating chains have not mixed.
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#r-hat

## Warning: Bulk Effective Samples Size (ESS) is too low, indicating posterior means and
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#bulk-ess

## Warning: Tail Effective Samples Size (ESS) is too low, indicating posterior variances
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#tail-ess

```

As we expected, there are warnings and bad mixing of the chains for $\tau_u[2]$; see also the traceplots in Figure 8.

```

print(fit_eeg_corr, pars = c("alpha", "beta", "tau_u", "sigma"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.

```

```
##
##          mean se_mean   sd  2.5%  25%  50%  75% 97.5% n_eff Rhat
## alpha    3.65    0.01 0.43  2.79  3.37  3.65  3.93  4.51 1554 1.00
## beta     2.33    0.01 0.62  1.15  1.92  2.32  2.75  3.54 4109 1.00
## tau_u[1] 2.19    0.01 0.37  1.54  1.93  2.16  2.42  3.00 2362 1.00
## tau_u[2] 1.65    0.11 0.86  0.29  0.97  1.59  2.25  3.49   61 1.07
## sigma    11.62   0.00 0.15 11.33 11.52 11.62 11.72 11.92 5615 1.00
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:40:33 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on
## convergence, Rhat=1).
```

```
traceplot(fit_eeg_corr, pars = c("alpha", "beta", "tau_u[1]", "tau_u[2]", "sigma"))
```

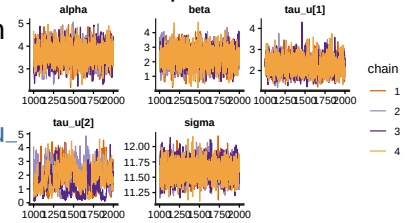


Figure 8: Traceplots of ‘alpha’, ‘beta’, ‘tau_u’, and ‘sigma’ from the model ‘fit_eeg_corr’.

The problem (which can also be discovered in a pairs plot) is the same one that we saw before: There is a strong dependency between `tau_u[2]` (in fact, $\log(\text{tau_u}[2])$), which is the parameter dimension that the sampler considers) and `u`, creating a funnel.

The solution to this problem is the reparameterization of this model. The parameterization for this type of model uses a Cholesky factorization. The mathematics and the intuition behind this parameterization are explained next.

7 Aside: Cholesky factorization

First, some definitions that we will need below. A matrix is square if the number of rows and columns is identical. A square matrix A is symmetric if $A^T = A$, i.e., if transposing the matrix gives the matrix back. Suppose that A is a known matrix with real numbers. If x is a vector of variables with length p (a $p \times 1$ matrix), then $x^T A x$ is called a quadratic form in x ($x^T A x$ will be a scalar, 1×1). If $x^T A x > 0$ for all x , then A is a positive definite matrix. If $x^T A x \geq 0$ for all $x \neq 0$, then A is positive semi-definite.

We encountered correlation matrices first in chapter 1. A correlation matrix is always symmetric, has ones along the diagonal, and real values ranging between -1 and 1 .

on the off-diagonals. Given a correlation matrix ρ_u , we can decompose it into a lower triangular matrix $\mathbf{L}_u$ such that $\mathbf{L}_u \mathbf{L}_u^T = \rho_u$. The matrix $\mathbf{L}_u$ is called the Cholesky factor of ρ_u . Intuitively, you can think of L_u as the matrix equivalent of the square root of ρ_u .

$$\mathbf{L}_u = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix} \quad (17)$$

For a model without a correlation between adjustments for the intercept and slope, we assumed that adjustments u_1 and u_2 were generated by two independent normal distributions. But now we want to allow the possibility that the adjustments can have a non-zero correlation. We can use the Cholesky factorization to generate correlated random variables in the following way.

1. Generate uncorrelated vectors, z_{u_1} and z_{u_2} , for each vector of adjustments u_1 and u_2 , as sampled from $Normal(0, 1)$:

$$z_{u_1} \sim Normal(0, 1)$$

$$z_{u_2} \sim Normal(0, 1)$$

2. By multiplying the Cholesky factor with the z 's, generate a matrix that contains two row vectors of correlated variables (with standard deviation 1).

$$\mathbf{L}_u \cdot \mathbf{z}_u = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} z_{u_1, subj=1} & z_{u_1, subj=2} & \dots & z_{u_1, subj=N_{subj}} \\ z_{u_2, subj=1} & z_{u_2, subj=2} & \dots & z_{u_2, subj=N_{subj}} \end{pmatrix}$$

$$\mathbf{L}_u \cdot \mathbf{z}_u = \begin{pmatrix} L_{11} \cdot z_{u_1,1} + 0 \cdot z_{u_2,1} & \dots & L_{11} \cdot z_{u_1, N_{subj}} \\ L_{21} \cdot z_{u_1,1} + L_{22} \cdot z_{u_2,1} & \dots & L_{21} \cdot z_{u_1, N_{subj}} + L_{22} \cdot z_{u_2, N_{subj}} \end{pmatrix}$$

A very informal explanation of why this works is that we are making the variable that corresponds to the slope to be a function of a scaled version of the intercept.

3. The last step is to scale the previous matrix to the desired standard deviation. We define the diagonalized matrix $diag_matrix(\tau_u)$ as before:

$$\begin{pmatrix} \tau_{u_1} & 0 \\ 0 & \tau_{u_2} \end{pmatrix}$$

Then pre-multiply it by the correlated variables with standard deviation 1 from before:

$$\begin{aligned} \mathbf{u} &= diag_matrix(\tau_u) \cdot \mathbf{L}_u \cdot \mathbf{z}_u = \\ &\begin{pmatrix} \tau_{u_1} & 0 \\ 0 & \tau_{u_2} \end{pmatrix} \begin{pmatrix} L_{11} \cdot z_{u_{1,1}} & \dots \\ L_{21} \cdot z_{u_{1,1}} + L_{22} \cdot z_{u_{2,1}} & \dots \end{pmatrix} \\ &\begin{pmatrix} \tau_{u_1} \cdot L_{11} \cdot z_{u_{1,1}} & \tau_{u_1} \cdot L_{11} \cdot z_{u_{1,2}} & \dots \\ \tau_{u_2} \cdot (L_{21} \cdot z_{u_{1,1}} + L_{22} \cdot z_{u_{2,1}}) & \tau_{u_2} \cdot (L_{21} \cdot z_{u_{1,2}} + L_{22} \cdot z_{u_{2,2}}) & \dots \end{pmatrix} \end{aligned}$$

7.1 Cholesky factorization in R

Let's assume a correlation of 0.8.

```
rho <- 0.8
# Correlation matrix
(rho_u <- matrix(c(1, rho, rho, 1), ncol = 2))

##      [,1] [,2]
## [1,]  1.0  0.8
## [2,]  0.8  1.0

# Cholesky factor:
# (Transpose it so that it looks the same as in Stan)
(L_u <- t(chol(rho_u)))

##      [,1] [,2]
## [1,]  1.0  0.0
## [2,]  0.8  0.6

# Verify that we recover rho_u,
# Recall that %*% indicates matrix multiplication
L_u %*% t(L_u)
```

```
##      [,1] [,2]
## [1,]  1.0  0.8
## [2,]  0.8  1.0
```

1. Generate uncorrelated z from a standard normal distribution assuming only 10 subjects.

```
N_subj <- 10
(z_u1 <- rnorm(N_subj, 0, 1))

## [1] -0.6113356  0.2519004 -1.0952038  0.4472544  0.4188446
## [6]  0.7905524  0.8469382 -1.1261633 -0.5493254  1.5185945

(z_u2 <- rnorm(N_subj, 0, 1))

## [1]  0.68825769 -0.18028047 -0.39543519 -2.78040974
## [5] -0.86248054  0.18599432 -0.26159309  0.69759283
## [9] -0.17833527 -0.06116634
```

2. Create matrix of correlated parameters.

First, create a matrix with the uncorrelated parameters:

```
# matrix z_u
(z_u <- matrix(c(z_u1, z_u2), ncol = N_subj, byrow = TRUE))

##      [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] -0.6113356  0.2519004 -1.0952038  0.4472544  0.4188446
## [2,]  0.6882577 -0.1802805 -0.3954352 -2.7804097 -0.8624805
##      [,6]      [,7]      [,8]      [,9]     [,10]
## [1,] 0.7905524  0.8469382 -1.1261633 -0.5493254  1.51859453
## [2,] 0.1859943 -0.2615931  0.6975928 -0.1783353 -0.06116634
```

Then, generate correlated parameters by pre-multiplying the z_u matrix with L_u .

```
L_u %*% z_u

##      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,] -0.61133558 0.25190037 -1.095204  0.4472544  0.4188446 0.7905524 0.8469382
## [2,] -0.07611385 0.09335202 -1.113424 -1.3104424 -0.1824126 0.7440385 0.5205947
##      [,8]      [,9]     [,10]
## [1,] -1.1261633 -0.5493254 1.518595
## [2,] -0.4823749 -0.5464615 1.178176
```

3. Use the following diagonal matrix to scale the z_u .

```
tau_u1 <- 0.2
tau_u2 <- 0.01
(diag_matrix_tau <- diag(c(tau_u1, tau_u2)))
```

```
##      [,1] [,2]
## [1,]  0.2 0.00
## [2,]  0.0 0.01
```

4. Finally, generate the adjustments for each subject u :

```
(u <- diag_matrix_tau %*% L_u %*% z_u)

##      [,1]      [,2]      [,3]      [,4]
## [1,] -0.1222671157 0.0503800748 -0.21904076 0.08945087
## [2,] -0.0007611385 0.0009335202 -0.01113424 -0.01310442
##      [,5]      [,6]      [,7]      [,8]
## [1,] 0.083768919 0.158110471 0.169387633 -0.225232655
## [2,] -0.001824126 0.007440385 0.005205947 -0.004823749
##      [,9]     [,10]
## [1,] -0.109865089 0.30371891
## [2,] -0.005464615 0.01178176
```

The rows are correlated, approximately 0.8

```
cor(u[1, ], u[2, ])
```

```
## [1] 0.6936171
```

The variance components can be recovered as well:

```
sd(u[1, ])
```

```
## [1] 0.1782751
```

```
sd(u[2, ])
```

```
## [1] 0.007895633
```

7.2 Application of the Cholesky factorization in Stan

The reparameterization of the model, which allows for a correlation between adjustments for the intercepts and slopes, in `hierarchical_corr2.stan` is shown below. The code implements the following new types and functions:

- `cholesky_factor_corr[2] L_u`, which defines `L_u` as a lower triangular (2×2) matrix which has to be the Cholesky factor of a correlation.
- `diag_pre_multiply(tau_u, L_u)` which makes a diagonal matrix out of the vector `tau_u` and multiplies it by `L_u`.
- `to_vector(z_u)` makes a long vector out the matrix `z_u`.
- `L_u ~ lkj_corr_cholesky(2)` is the Cholesky factor associated with the LKJ correlation distribution. It implies that `L_u * L_u' ~ lkj_corr(2.0)`. The symbol `'` indicates transposition (in R, this is the function `t()`).

```
data {
  int<lower=1> N;
  vector[N] signal;
  int<lower = 1> N_subj;
  vector[N] c_cloze;
  array[N] int<lower = 1, upper = N_subj> subj;
}

parameters {
  real<lower = 0> sigma;
  vector<lower = 0>[2] tau_u;
  real alpha;
  real beta;
  matrix[2, N_subj] z_u;
  cholesky_factor_corr[2] L_u;
}

transformed parameters {
  matrix[N_subj, 2] u;
  u = (diag_pre_multiply(tau_u, L_u) * z_u)';
}

model {
  target += normal_lpdf(alpha | 0, 10);
  target += normal_lpdf(beta | 0, 10);
  target += normal_lpdf(sigma | 0, 50) -
    normal_lccdf(0 | 0, 50);
  target += normal_lpdf(tau_u | 0, 20) -
    2 * normal_lccdf(0 | 0, 20);
}
```



```

target += lkj_corr_cholesky_lpdf(L_u | 2);
target += std_normal_lpdf(to_vector(z_u));
target += normal_lpdf(signal | alpha + u[subj, 1] +
                      c_cloze .* (beta + u[subj, 2]), sigma);
}
generated quantities {
  matrix[2, 2] R_u = (L_u * L_u');
  real rho_u = R_u[1, 2];
  vector[N_subj] effect_by_subj = beta + u[ , 2];
}

```

In this Stan model, we also defined an `effect_by_subject` in the generated quantities. This would allow us to plot or to summarize by-subject effects of cloze probability. This is very useful when studying individual differences.⁴

One can recover the correlation parameter by adding in the generated quantities section code to extract one element of the 2×2 correlation matrix. The correlation matrix is recovered with `L_u * L_u'`, and then `[1, 2]` extracts the element in the first row and second column (recall that the diagonal is filled with ones).

Fit the new model:

```

hierarchical_corr2 <- system.file("stan_models",
                                "hierarchical_corr2.stan",
                                package = "bcogsci")
fit_eeg_corr2 <- stan(hierarchical_corr2, data = ls_eeg)

```

The Cholesky matrix has some elements which are always zero or one, and thus the variance within and between chains (and therefore `Rhat`) are not defined. However, the rest of the parameters of the model have an appropriate number of effective sample size (more than 10% of the total number of post-warmup samples), `Rhats` are close to one, and the chains are mixing well; see also the traceplots in Figure 9.

```

print(fit_eeg_corr2,
      pars =
        c("alpha", "beta", "tau_u", "rho_u", "sigma", "L_u"))

```

⁴ For an example, see: Himanshu Yadav, Dario Paape, Garrett Smith, Brian W. Dillon, and Shravan Vasishth. Individual differences in cue weighting in sentence comprehension: An evaluation using Approximate Bayesian Computation. Open Mind, 2022.

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##          mean se_mean   sd  2.5%  25%   50%   75%  97.5% n_eff Rhat
## alpha      3.64    0.01 0.42   2.82  3.36  3.64  3.92  4.50  1190    1
## beta       2.32    0.01 0.60   1.12  1.92  2.33  2.72  3.48  3414    1
## tau_u[1]    2.21    0.01 0.38   1.56  1.95  2.18  2.44  3.03  1531    1
## tau_u[2]    1.63    0.03 0.90   0.12  0.94  1.59  2.24  3.46   940    1
## rho_u       0.17    0.01 0.34  -0.53 -0.05  0.19  0.41  0.80  3429    1
## sigma     11.62    0.00 0.16  11.31 11.51 11.62 11.72 11.93  6437    1
## L_u[1,1]    1.00     NaN 0.00   1.00  1.00  1.00  1.00  1.00   NaN   NaN
## L_u[1,2]    0.00     NaN 0.00   0.00  0.00  0.00  0.00  0.00   NaN   NaN
## L_u[2,1]    0.17    0.01 0.34  -0.53 -0.05  0.19  0.41  0.80  3429    1
## L_u[2,2]    0.92    0.00 0.11   0.59  0.89  0.96  0.99  1.00  1554    1
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:41:04 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
traceplot(fit_eeg_corr2,
  pars = c("alpha", "beta", "tau_u", "L_u[2,1]"
```

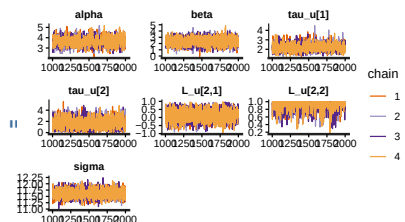


Figure 9: Traceplots of 'alpha', 'beta', 'tau_u', 'L_u', and 'sigma' from the model 'fit_eeg_corr'.

Is there a correlation between the by-subject intercept and slope?

Let's visualize some of the posteriors with the following code (see Figure 10):

```
mcmc_hist(as.data.frame(fit_eeg_corr2),
  pars = c("beta", "rho_u"))
```

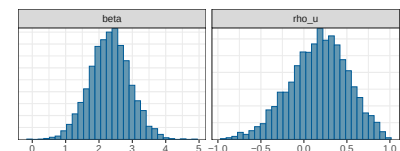


Figure 10: Histograms of the samples of the posteriors of 'beta' and 'rho_u' from the model 'fit_eeg_corr2'.

Figure 10 shows that the posterior distribution is widely spread out between -1 and $+1$. One can't really learn from these data whether the by-subject intercepts and slopes are correlated. The broad spread of the posterior indicates that we don't have enough data to estimate this parameter with high enough precision: the posterior is basically just reflecting the prior specification (the LKJcorr prior with parameter $\eta = 2$).

8 By-subject and by-items correlated varying intercept varying slopes model

We extend the previous model by adding by-items intercepts and slopes, and priors and hyperpriors that reflect the potential correlation between by-items adjustments to intercepts and slopes:

$$\begin{aligned} \text{signal}_n \sim \text{Normal}(\alpha + u_{\text{subj}[n],1} + w_{\text{item}[n],1} + \\ c_{\text{cloze}_n} \cdot (\beta + u_{\text{subj}[n],2} + w_{\text{item}[n],2}), \sigma) \end{aligned} \quad (18)$$

The correlation is indicated in the priors on the adjustments for the vectors representing the varying intercepts u_1 and varying slopes u_2 for subjects, and the varying intercepts w_1 and varying slopes w_2 for items.

- Priors:

$$\begin{aligned} \alpha &\sim \text{Normal}(0, 10) \\ \beta &\sim \text{Normal}(0, 10) \\ \sigma &\sim \text{Normal}_+(0, 50) \\ \begin{pmatrix} u_{i,1} \\ u_{i,2} \end{pmatrix} &\sim \mathcal{N} \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Sigma_u \right) \\ \begin{pmatrix} w_{i,1} \\ w_{i,2} \end{pmatrix} &\sim \mathcal{N} \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Sigma_w \right) \end{aligned} \quad (19)$$

$$\Sigma_u = \begin{pmatrix} \tau_{u_1}^2 & \rho_u \tau_{u_1} \tau_{u_2} \\ \rho_u \tau_{u_1} \tau_{u_2} & \tau_{u_2}^2 \end{pmatrix} \quad (20)$$

$$\Sigma_w = \begin{pmatrix} \tau_{w_1}^2 & \rho_w \tau_{w_1} \tau_{w_2} \\ \rho_w \tau_{w_1} \tau_{w_2} & \tau_{w_2}^2 \end{pmatrix} \quad (21)$$

$$\begin{aligned} \tau_{u_1} &\sim \text{Normal}_+(0, 20) \\ \tau_{u_2} &\sim \text{Normal}_+(0, 20) \\ \rho_u &\sim \text{LKJcorr}(2) \end{aligned} \quad (22)$$

$$\begin{aligned} \tau_{w_1} &\sim \text{Normal}_+(0, 20) \\ \tau_{w_2} &\sim \text{Normal}_+(0, 20) \\ \rho_w &\sim \text{LKJcorr}(2) \end{aligned} \quad (23)$$

The translation to Stan (`hierarchical_corr_by.stan`) looks as follows:

```
data {
  int<lower=1> N;
  vector[N] signal;
  int<lower = 1> N_subj;
  int<lower = 1> N_item;
  vector[N] c_cloze;
  array[N] int<lower = 1, upper = N_subj> subj;
  array[N] int<lower = 1, upper = N_item> item;
}

parameters {
  real<lower = 0> sigma;
  vector<lower = 0>[2] tau_u;
  vector<lower = 0>[2] tau_w;
  real alpha;
  real beta;
  matrix[2, N_subj] z_u;
  matrix[2, N_item] z_w;
  cholesky_factor_corr[2] L_u;
  cholesky_factor_corr[2] L_w;
}

transformed parameters {
  matrix[N_subj, 2] u;
  matrix[N_item, 2] w;
  u = (diag_pre_multiply(tau_u, L_u) * z_u)';
  w = (diag_pre_multiply(tau_w, L_w) * z_w)';
}

model {
  target += normal_lpdf(alpha | 0,10);
  target += normal_lpdf(beta | 0,10);
  target += normal_lpdf(sigma | 0, 50) -
    normal_lccdf(0 | 0, 50);
  target += normal_lpdf(tau_u | 0, 20) -
    2 * normal_lccdf(0 | 0, 20);
  target += normal_lpdf(tau_w | 0, 20) -
    2 * normal_lccdf(0 | 0, 20);
  target += lkj_corr_cholesky_lpdf(L_u | 2);
}
```

```

target += lkj_corr_cholesky_lpdf(L_w | 2);
target += std_normal_lpdf(to_vector(z_u));
target += std_normal_lpdf(to_vector(z_w));
target += normal_lpdf(signal | alpha + u[subj, 1] + w[item, 1] +
                      c_cloze .* (beta + u[subj, 2] + w[item, 2]),
                      sigma);
}
generated quantities {
  real rho_u = (L_u * L_u')[1, 2];
  real rho_w = (L_w * L_w')[1, 2];
}

```

Add item as a number to the data and store it in a list:

```

df_eeg <- df_eeg %>%
  mutate(item = as.numeric(as.factor(item)))
ls_eeg <- list(
  N = nrow(df_eeg),
  signal = df_eeg$signal,
  c_cloze = df_eeg$c_cloze,
  subj = df_eeg$subj,
  item = df_eeg$item,
  N_subj = max(df_eeg$subj),
  N_item = max(df_eeg$item)
)

```

Fit the model:

```

hierarchical_corr_by <- system.file("stan_models",
                                     "hierarchical_corr_by.stan",
                                     package = "bcogsci")
fit_eeg_corr_by <- stan(hierarchical_corr_by, data = ls_eeg)

```

9 Custom distributions in Stan

Stan includes a large number of distributions, but what happens if we need a distribution that is not provided? In many cases, we can simply build a custom distribution by combining the ever-growing number of functions available in the Stan language.

9.1 A change of variables with the reciprocal normal distribution

In previous chapters, when faced with response times, we assumed a log-normal distribution. The log-normal distribution moves the inferences relating to the location parameter into a multiplicative frame. Another alternative, however, is to assume a “reciprocal”-normal distribution of response times. This is referred to hereafter as *rec-normal* in the text, and as the *RecNormal* distribution in equations. That is, we may want to assume that the reciprocal of the response times are normally distributed. This assumption may be theoretically motivated or it may arise from the application of the Box-Cox variance stabilizing transform procedure.

$$\begin{aligned} 1/y &\sim \text{Normal}(\mu, \sigma) \\ y &\sim \text{RecNormal}(\mu, \sigma) \end{aligned} \quad (\#eq : \text{recip}) \quad (24)$$

An interesting aspect of the *rec-normal* is that it affords an interpretation of the location parameter in terms of rate or speed rather than time. Analogously to the case of the log-normal, neither the location μ nor the scale σ are in the same scale as the dependent variable y . These parameters are in the same scale as the transformed dependent variable (here, $1/y$) that is assumed to be normally distributed.

By setting $\mu = 0.002$ and $\sigma = 0.0004$, data can be generated from the *rec-normal* that looks right-skewed and not unlike a distribution of response times. The code below shows some summary statistics of the generated data.

```
N <- 100
mu <- 0.002
sigma <- 0.0004
rt <- 1 / rnorm(N, mu, sigma)
c(mean(rt), sd(rt), min(rt), max(rt))
## [1] 523.8931 107.9348 325.9173 902.4572
```

Figure 11 shows the distribution generated with the code shown above.

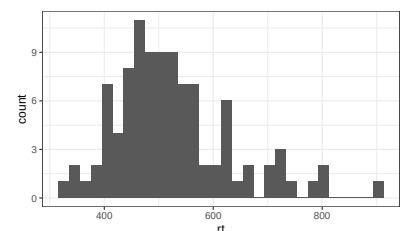


Figure 11: Distribution of synthetic data with a *rec-normal* distribution with parameters $\mu = 0.002$ and $\sigma = 0.0004$.

We can fit the rec-normal distribution to the response times in a simple model with a normal likelihood, by storing the reciprocal of the response times ($1/RT$) in the vector variable `recRT` (rather than storing the “raw” response times):

```
model {
  \\ priors go here
  target += normal_lpdf(recRT | mu, sigma);
}
```

One issue here is that the parameters of the likelihood, μ and σ are going to be very far away from the unit scale (0.002 and 0.0004 respectively). Due to the way Stan’s sampler is built, parameters that are too small (much smaller than 1) or too large (much larger than 1) can cause convergence problems. A straightforward solution is to fit the normal distribution to parameters in reciprocal of seconds rather than milliseconds; this would make the parameters have values that are 1000 times larger (2 and 0.4 respectively). We can do this using the transformed parameters block. Although we use μ and σ to fit the data, the priors are defined on the parameters μ_s and σ_s (μ_s and σ_s).

Unless one can rely on previous estimates, finding good priors for μ_s and σ_s is not trivial. To define appropriate priors, we would need to start with relatively arbitrary priors, inspect the prior predictive distributions, adjust the priors, and repeat the inspection of prior predictive distributions until these distributions start to look realistic. In the interest of conserving space, we skip this iterative process here, and assign the following priors:

$$\begin{aligned}\mu_s &\sim \text{Normal}(2, 1) \\ \sigma_s &\sim \text{Normal}_+(0.4, 0.2)\end{aligned}\tag{25}$$

This model is implemented in the file `normal_recrt.stan`, available in the `bcogsci` package:

```
data {
  int<lower = 1> N;
```

```

    vector[N] recRT;
  }
  parameters {
    real mu_s;
    real<lower = 0> sigma_s;
  }
  transformed parameters {
    real mu = mu_s / 1000;
    real sigma = sigma_s / 1000;
  }
  model {
    target += normal_lpdf(mu_s | 2, 1);
    target += normal_lpdf(sigma_s | 0.4, 0.2) +
      - normal_lccdf(0 | 0.4, 0.2);
    target += normal_lpdf(recRT | mu, sigma);
  }

```

Fit and display the summary of the previous model:

```

normal_recrt <- system.file("stan_models",
                           "normal_recrt.stan",
                           package = "bcogsci")
fit_rec <- stan(normal_recrt, data = list(N = N, recRT = 1 / rt))
print(fit_rec, pars = c("mu", "sigma"), digits = 4)

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##           mean se_mean sd   2.5%   25%   50%   75%  97.5% n_eff  Rhat
## mu       2e-03      0  0 0.0019 2e-03 2e-03 2e-03 0.0021  3357 1.0001
## sigma 4e-04      0  0 0.0003 4e-04 4e-04 4e-04 0.0004  3389 1.0003
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:42:07 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).

```

Is a rec-normal likelihood more appropriate than a log-normal likelihood? As things stand, we cannot compare

the models with these two likelihoods. This is because the dependent variables are different: we cannot compare reciprocal response times with untransformed response times on the millisecond scale. Model comparison with the Bayes factor or cross-validation can only compare models with the same dependent variables; we discuss model comparison later.

If we do want to compare the reciprocal-normal likelihood and the log-normal likelihood, we have to set up the models with the two likelihoods in such a way that the dependent measure is on the raw millisecond scale in each model. This means that for the reciprocal normal likelihood, the model will receive as data raw reading times in milliseconds, and these will be treated as a transformed random variable from reciprocal reading times. This approach is discussed next, but requires knowledge of the **Jacobian adjustment**.

9.2 *Scaling a probability density with the Jacobian adjustment*

To work with the original dependent variable (RT rather than $1/RT$) we need to perform a *change of variables*: If the random variable X represents the reciprocal reading times ($1/RT$), we can transform this random variable to a new one, $Y = 1/X = 1/(1/RT) = RT$, yielding a transformed random variable Y which represents reading time.

This change of variables requires an adjustment to the (unnormalized log) posterior probability to account for the distortion caused by the transform.⁵ The probability must be scaled by a *Jacobian* adjustment, which, in the univariate case as this one, is the absolute value of the derivative of the transform.⁶ The absolute value of any number is represented by enclosing it in two vertical bars: e.g., $|-2| = 2$.

⁵ Not every transformation is valid, univariate changes of variables must be monotonic and differentiable, multivariate changes of variables must be surjective and differentiable.

⁶ In the multivariate case, it is equal to the absolute determinant of the Jacobian, the matrix of all its first-order partial derivatives, of the transform; references in the book.

$$\begin{aligned}
p(RT_n|\mu, \sigma) &= \text{Normal}(1/RT_n|\mu, \sigma) \left| \frac{d}{dRT_n} 1/RT_n \right| = \\
&\text{Normal}(1/RT_n|\mu, \sigma) \cdot \left| -1/RT_n^2 \right| = \\
&\text{Normal}(1/RT_n|\mu, \sigma) \cdot 1/RT_n^2
\end{aligned} \tag{26}$$

If we omit the Jacobian adjustment, we are essentially fitting a different and unnormalized probability density function. The discrepancy between the correct and incorrect posterior would depend on how large the Jacobian adjustment for our model is.

Because Stan works in log space, rather than multiplying the Jacobian adjustment, we add its logarithm to the log-probability density ($\log(1/RT_n^2) = -2 \cdot \log(RT_n)$). The contribution to the log likelihood (of μ and σ) with its adjustment based from an individual observation, RT_n , would be as follows.

$$\log(\text{Normal}(1/RT_n|\mu, \sigma)) - 2 \cdot \log(RT_n) \tag{27}$$

We obtain the log likelihood based on all the N observations by summing the log likelihood of individual observations.

$$\log \mathcal{L} = \sum_n^N \log(\text{Normal}(1/RT_n|\mu, \sigma)) - \sum_n^N 2 \cdot \log(RT_n) \tag{28}$$

In Stan, this summing up is done as follows. The function `normal_lpdf` applied to a vector already returns the sum of the individual log-probability densities. The Jacobian adjustment $2 \cdot \log(RT)$, which returns a vector of values, has to be summed up and added in manually (the term therefore becomes `-sum(2*log(RT))`).

```
target += normal_lpdf(1 ./ RT | mu, sigma)
        - sum(2 * log(RT));
```

Before fitting the model with the change of variables, we are going to truncate the distribution. We didn't encounter negative values in our synthetic data, but this

was because the distribution was not too spread out; that is, the scale was much smaller than the location ($\sigma \ll \mu$). However, in principle we could end up generating negative values. For this reason, we truncate the underlying normal distribution. The reciprocal truncated normal distribution has been argued to be an appropriate model of response times, neural inter-spike intervals, and latency distributions of saccades in a simple optimality model in which reward is maximized to yield an optimal response rate.

Because we have N observations, the truncation consists of adding $- N * \text{normal_lccdf}(0 \mid \mu, \sigma)$ to target. The complete model, `recnormal_rt.stan`, including the truncation is as follows:

```
data {
  int<lower = 1> N;
  vector[N] RT;
}
parameters {
  real mu_s;
  real<lower = 0> sigma_s;
}
transformed parameters {
  real mu = mu_s / 1000;
  real sigma = sigma_s / 1000;
}
model {
  target += normal_lpdf(mu_s | 2, 1);
  target += normal_lpdf(sigma_s | 0.4, 0.2)
    - normal_lccdf(0 | 0.4, 0.2);
  target += normal_lpdf(1 ./ RT | mu, sigma)
    - N * normal_lccdf(0 | mu, sigma)
    - sum(2 * log(RT));
}
```

Next, generate data from a reciprocal truncated normal distribution:

```
N <- 100
```

```
mu <- .002
sigma <- .0004
rt <- 1 / rtnorm(N, mu, sigma, a = 0)
```

Fit the model to the data:

```
recnormal_rt <- system.file("stan_models",
  "recnormal_rt.stan",
  package = "bcogsci")
fit_rec <- stan(recnormal_rt, data = list(N = N, RT = rt))
```

Print the posterior summary:

```
print(fit_rec, pars = c("mu", "sigma"), digits = 4)

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##           mean se_mean sd  2.5%   25%   50%   75%  97.5% n_eff  Rhat
## mu      2e-03      0  0 2e-03 2e-03 2e-03 0.0021 0.0021  3205 1.0002
## sigma 4e-04      0  0 3e-04 4e-04 4e-04 0.0004 0.0004  2964 1.0001
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:42:27 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

We get the same results as before, but now we could potentially compare this model to one that assumes a log-normal likelihood, for example.

An important question is the following: Does every transformation of a random variable require a Jacobian adjustment? Essentially, if we assign a distribution to a random variable and then transform it, this does not require a Jacobian adjustment.

Alternatively, if we apply a transformation on a random variable first, and then assign a distribution to the transformed random variable afterwards, we have a change of variables. This requires a Jacobian adjustment. This is the reason why $1 ./ RT$ requires a Jacobian adjustment, but not μ_s , μ , σ_s , or σ in the previous model.

As a last step, we can encapsulate our new distribution in a function, and also create a random number generator function. This is done in a block called `functions`. To create a function, we need to specify the type of every argument and the type that the function returns.

As a simple example, if we would like a function to center a vector, we would do it as follows:

```
functions {
  vector center(vector x) {
    vector[num_elements(x)] centered;
    centered = x - mean(x);
    return centered;
  }
}
data {
  ...
}
parameters {
  ...
}
model {
  ...
}
```

We want to create a `log(PDF)` function, similar to the native Stan functions that end in `_lpdf`. Our function will take as arguments a vector `RT`, and real numbers `mu` and `sigma`. In `_lpdf` functions, (some of) the arguments (e.g., `RT`, `mu`, and `sigma`) can be vectorized, but the output of the function is always a real number: the sum of the `log(PDF)` evaluated at every value of the random variable at the left of the pipe. As we show below, to do that we just move the right hand side of target in our original model inside our new function. See the section entitled *Further Reading* at the end of this chapter for more about Stan functions.

For our custom random number generator function (which should end with `_rng`), we have the added complication that the function is truncated. We simply generate

random values from a normal distribution, do the reciprocal transformation and if the number is above zero, this number is returned; otherwise, a new number is drawn. This implementation may be inefficient when rejections occur frequently. Another, more complex implementation using the inverse CDF approach can be found in section 20.10 of the Stan User's guide.

The complete code can be found in `recnormal_rt.f.stan`, and is also shown below:

```
functions {
  real recnormal_lpdf(vector y, real mu, real sigma){
    real lpdf;
    lpdf = normal_lpdf(1 ./ y | mu, sigma)
    - num_elements(y) * normal_lccdf(0 | mu, sigma)
    - sum(2 * log(y));
    return lpdf;
  }
  real recnormal_rng(real mu, real sigma){
    real y = 0;
    while (y <= 0)
      y = 1 / normal_rng(mu, sigma);
    return y;
  }
}

data {
  int<lower = 1> N;
  vector<lower = 0>[N] RT;
}

parameters {
  real mu_s;
  real<lower = 0> sigma_s;
}

transformed parameters {
  real mu = mu_s / 1000;
  real sigma = sigma_s / 1000;
}

model {
  target += normal_lpdf(mu_s | 2, 1);
}
```

```

    target += normal_lpdf(sigma_s | 0.4, 0.2)
             - normal_lccdf(0 | 0.4, 0.2);
    target += recnormal_lpdf(RT | mu, sigma);
  }
generated quantities {
  array[N] real rt_pred;
  for (n in 1:N)
    rt_pred[n] = recnormal_rng(mu, sigma);
}

```

Fit the model to the simulated data:

```

recnormal_rt_f <- system.file("stan_models",
                             "recnormal_rt_f.stan",
                             package = "bcogsci")
fit_rec_f <- stan(recnormal_rt_f, data = list(N = N, RT = rt))

```

Print the summary:

```

print(fit_rec_f, pars = c("mu", "sigma"), digits = 4)

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##           mean se_mean sd  2.5%   25%   50%   75%  97.5% n_eff  Rhat
## mu       2e-03      0  0 2e-03 2e-03 2e-03 0.0021 0.0021  2790 1.0005
## sigma 4e-04      0  0 3e-04 4e-04 4e-04 0.0004 0.0004  3022 1.0008
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:42:48 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).

```

9.3 Validation of a computed posterior distribution

The model converged, but did it work? At the very least we expect that the simulated data that we used to test the model shows similar distribution to the posterior predictive distributions. This is because we are fitting the data with the same function that we use to generate the data.

Figure 12 shows that the simulated data and the posterior predictive distributions are similar.

```
ppc_dens_overlay(rt, yrep = extract(fit_rec_f)$rt_pred[,
  coord_cartesian(xlim = c(0, 2000))
```

Our synthetic data set was generated by first defining a *ground truth* vector of parameters $[\mu; \sigma]$. We would expect that the true values of the parameters μ and σ should be well inside the bulk of the posterior distribution of our model. We investigate this by plotting the true values of the parameters together with their posterior distributions using the function `mcmc_recover_hist` of `bayesplot` as shown below in Figure 13.

```
post_rec <- as.data.frame(fit_rec_f)[,c(1,2)]
```

```
mcmc_recover_hist(post_rec, true = c(mu, sigma))
```

Even though this approach can capture serious misspecifications in our models, it has three important shortcomings.

First, it's unclear what we should conclude if the true value of a parameter is not in the bulk of its posterior distribution. Even in a well-specified model, if we inspect enough parameters we will find that for some parameters, the bulks of their respective marginal posterior distributions can be relatively far away from their true values.

The second shortcoming is that if the posteriors that we obtain are very wide, it might not be clear what to consider the bulk of the posterior distribution that needs to be inspected.

The third shortcoming is that regardless of how a successful recovery of parameters is defined, we might be able to recover a posterior based on data generated from some parts of the parameter space, but not based on data generated from other parts of parameter space. However, inspecting the entire parameter space is infeasible.

An alternative to our approach of plotting the true values of the parameters together with their posterior distri-

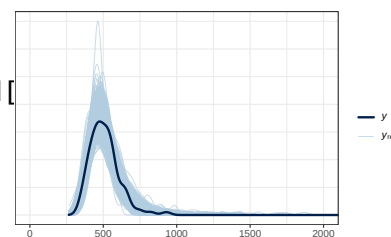


Figure 12: A posterior predictive check in comparison with the simulated data.

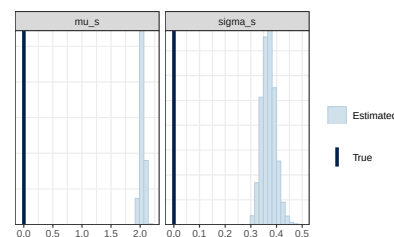


Figure 13: Posterior distributions of the parameters together with their true values.

butions is simulation-based calibration, but we will skip this topic in these lectures. See the textbook for details.

Print the summary:

```
print(fit_eeg_corr_by,
      pars = c("alpha", "beta", "sigma", "tau_u", "tau_w",
               "rho_u", "rho_w"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##          mean se_mean   sd  2.5%  25%   50%   75%  97.5% n_eff Rhat
## alpha      3.67    0.01 0.47  2.74  3.36  3.66  3.99  4.58  2727 1.00
## beta       2.30    0.01 0.68  0.96  1.85  2.30  2.74  3.61  4962 1.00
## sigma     11.49    0.00 0.16 11.19 11.38 11.48 11.59 11.80  5825 1.00
## tau_u[1]   2.21    0.01 0.37  1.55  1.95  2.18  2.45  3.02  1357 1.00
## tau_u[2]   1.46    0.02 0.87  0.08  0.76  1.41  2.09  3.25  1275 1.00
## tau_w[1]   1.51    0.01 0.35  0.80  1.28  1.51  1.74  2.18  1183 1.01
## tau_w[2]   2.34    0.03 1.01  0.27  1.68  2.38  3.02  4.30  1271 1.00
## rho_u      0.13    0.01 0.36 -0.62 -0.12  0.15  0.38  0.76  5036 1.00
## rho_w     -0.41    0.01 0.31 -0.90 -0.64 -0.46 -0.23  0.34  2486 1.00
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:41:45 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

The summary above shows that the data are far too sparse to get tight estimates of the correlation parameters $\rho_u[1,2]$ and $\rho_w[1,2]$. Both posteriors are widely spread out.

This completes our review of hierarchical models and their implementation in Stan. The importance of coding a hierarchical model directly in Stan rather than using `brms` is that this increases the flexibility of the type of models that we can fit. In fact, we will see in later lectures that the same “machinery” can be used to have hierarchical parameters in cognitive models.